

DBMS PLACEMENT PREPARATION NOTES

Modules 18–21 • Indexing • B-Trees & B+ Trees • Hashing • Views, Procedures, Functions & Triggers

Continuation: These notes follow serializability, locking, timestamp ordering and recovery. They focus on database access structures and common SQL/DBMS objects used in placement interviews and MCQs.

MODULE 18 — INDEXING

Index: A separate data structure that helps the DBMS find rows faster without scanning every row of a table.



Index speeds lookup by avoiding a full table scan in suitable queries.

1. Why Indexing?

Suppose a table has one million rows and the query searches for `Employee_ID = 1050`. Without a useful index, the DBMS may need to inspect many rows. An index can guide the search directly toward matching records.

2. Index Key

Index key: The column or set of columns used by the index, such as `Employee_ID` or `(Department_ID, Salary)`.

3. Primary Index

An index created around the ordering/key of a table's primary key. In traditional textbook terminology, it is associated with an ordered data file whose search key is the primary key.

4. Clustering Index

An index whose search-key order corresponds to the physical ordering of records. A table generally has only one physical ordering, so there is typically one clustering order.

5. Secondary Index

An additional index on a field that is not the table's primary physical ordering. Multiple secondary indexes can exist.

6. Dense vs Sparse Index

Dense Index	Sparse Index
Has an index entry for every search-key value/record as defined by the index.	Has only some search-key values, often one per block in ordered-file.
Faster lookup but more index space.	Less space, but may require additional searching after locating a nearby entry.

7. Clustered vs Non-Clustered — Practical Meaning

Clustered: table data is organized around the index order. **Non-clustered:** index is separate from the physical row order and points to the data.

8. Composite Index

An index built using more than one column, for example **(department_id, salary)**. Column order matters because a composite index is most useful for searches that use its leading columns.

9. Index Advantages

Faster equality/range searches, faster joins or ordering in suitable cases, and reduced need for full table scans.

10. Index Disadvantages

Consumes storage and can slow INSERT, UPDATE and DELETE because index entries may also need to be maintained.

11. When an Index May Not Help

A query that returns most of the table, a very small table, or a condition that prevents effective use of the index may still require a scan. The optimizer decides based on cost and statistics.

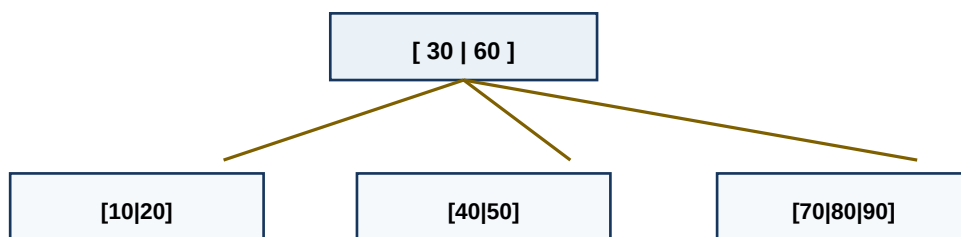
■ Placement memory

Index = faster reads, extra storage + maintenance cost. Composite index = column order matters.

MODULE 19 — B-TREE & B+ TREE INDEXING

B-tree: A balanced multiway search tree designed to keep many keys per node and reduce disk I/O.

B+ tree: A B-tree variant where actual record pointers/data entries are kept at leaf level, while internal nodes mainly guide searches.



Balanced search tree: keys guide the search toward the correct child.

1. Why Trees for Databases?

Database tables can be much larger than memory. Tree indexes keep the height small, so finding a key usually needs only a small number of node accesses.

2. Balanced Tree

Balanced: Leaf nodes are kept at the same level in standard B/B+ tree designs. This prevents one search path from becoming much longer than another.

3. B-Tree Properties — Placement Level

Nodes can contain multiple keys and multiple children. The tree remains balanced after insertions and deletions through operations such as splitting and merging/redistribution.

4. B+ Tree Structure

Internal nodes contain separator/search keys and child pointers. Leaf nodes contain search keys with record pointers (or the records, depending on implementation). Leaves are commonly linked for efficient sequential/range traversal.

5. Why B+ Trees Are Popular

High fan-out keeps the tree shallow, and linked leaves make range queries efficient.

6. Search

Start at the root, compare the target key with separator keys, follow the appropriate child, and continue until the leaf containing the target is reached.

7. Insertion

Insert into the appropriate leaf. If the node overflows, split it and propagate a separator key upward. Splitting can continue toward the root.

8. Deletion

Remove the key. If a node falls below its allowed occupancy, the tree may redistribute keys with a sibling or merge nodes, with changes propagated upward when needed.

9. B-Tree vs B+ Tree

B-Tree	B+ Tree
Records/pointers may appear in internal nodes as well as leaf nodes.	Data/record pointers are kept at leaves in the common B+ design.
Search may finish at an internal node.	Search generally reaches a leaf.
Sequential/range traversal is less naturally supported.	Linked leaves make range traversal efficient.
Less common as a database index than B+ tree.	Widely used for database indexes and storage engines.

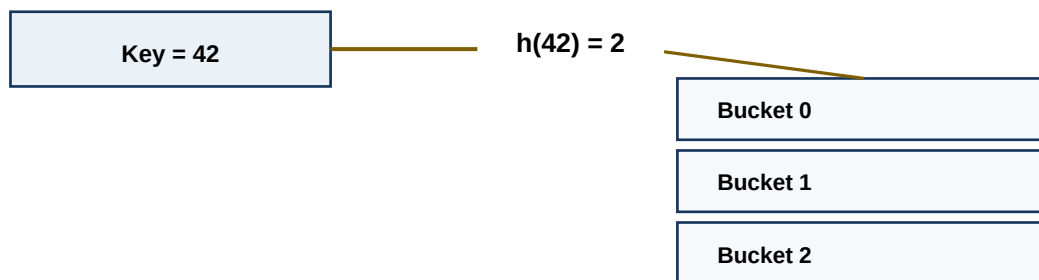
■ Placement memory

B+ tree → all data entries at leaves + linked leaves + excellent range queries.

Balanced tree → small height → fewer disk I/Os.

MODULE 20 — HASHING IN DBMS

Hashing: A technique that uses a hash function to map a search key to a bucket/address.



Hash function maps a key to a bucket/address.

1. Hash Function

Hash function $h(\text{key})$ converts a key into a bucket number. Example: $h(42) = 2$.

2. Bucket

A **bucket** is a storage location or group of locations where records mapped to the same hash value are stored.

3. Collision

Collision: Two different keys produce the same hash value/bucket.

Example: $h(12)=2$ and $h(22)=2$. Both keys map to bucket 2.

4. Collision Handling

Common ideas include **overflow buckets/chaining**, where extra storage is linked to a bucket when it becomes full. The exact technique depends on the hashing implementation.

5. Static Hashing

The number of primary buckets is fixed. As data grows, overflow chains may become longer and performance may degrade.

6. Dynamic Hashing

The hash structure can grow or reorganize as the amount of data changes. Common database techniques include **extendible hashing** and **linear hashing**.

7. Hash Index vs B+ Tree

Hashing	B+ Tree
Excellent for equality search such as key = 100.	Supports equality and efficient range searches.
Poor for ordered/range traversal.	Good for ORDER BY and range conditions in suitable cases.
Collision/overflow must be managed.	Maintains balanced ordered structure.
Does not naturally preserve key order.	Keys remain ordered across the tree/leaves.

8. Load Factor — Basic Concept

Load factor: A measure of how full a hash structure is. A high load factor can increase collisions and overflow.

9. When Hashing Is Useful

Hashing is especially useful for equality lookups where ordering is not required.

■ Placement memory

Hashing → equality search. **B+ tree** → equality + range/ordered access. **Collision** = different keys map to same bucket.

MODULE 21 — VIEWS, STORED PROCEDURES, FUNCTIONS & TRIGGERS

These are database objects/features that help organize reusable SQL logic, control access, and automate actions.



Reusable database objects: stored logic and controlled data access.

1. VIEW

View: A virtual table defined by a query. It normally stores the query definition rather than a separate copy of all underlying data.

Example:

```
CREATE VIEW high_salary AS SELECT name, salary FROM Employee WHERE salary > 50000;
```

Query it like a table:

```
SELECT * FROM high_salary;
```

Why views are used: simplify complex queries, hide unnecessary columns, and provide a controlled way to expose data.

2. View vs Table

Table	View
Stores table data.	Usually represents data through a stored query definition.
Independent base relation.	Depends on underlying tables/queries.
Can be directly populated/modified subject to constraints.	Modification depends on the view definition and DBMS rules.

3. STORED PROCEDURE

Stored procedure: A named set of SQL/procedural statements stored in the database and executed when called.

A procedure can accept parameters and can perform multiple operations. Exact syntax differs by DBMS.

Conceptual example:

```
CREATE PROCEDURE GetEmployee(IN emp_id INT) ...
```

4. FUNCTION

Database function: A stored program that normally returns a value and can be used in expressions, depending on the DBMS.

Example idea: calculate tax, convert a value, or return a derived result.

5. Procedure vs Function

Procedure	Function
Used to perform a set of operations.	Designed to return a value.
May have input/output parameters depending on DBMS.	Returns a value and can often be used in expressions.
Invocation syntax is DBMS-specific.	Invocation and restrictions are DBMS-specific.

6. TRIGGER

Trigger: A database program that automatically executes when a specified event occurs, such as INSERT, UPDATE, or DELETE.

Important terminology: **Event** = operation that activates the trigger. **Timing** = BEFORE/AFTER in systems that support it. **Row-level** = trigger can execute for each affected row.

Example use: automatically record an audit entry after an employee salary update.

7. Trigger Example — Concept

UPDATE Employee SET salary = 60000 WHERE emp_id = 10; → trigger fires → audit table receives a record.

8. Trigger vs Procedure

Trigger	Procedure
Runs automatically when its defined event occurs.	Runs when explicitly called/invoked.
Associated with table/view events depending on DBMS.	Independent stored routine.
Useful for auditing, automatic validation/actions.	Useful for reusable business/database operations.

9. BEFORE vs AFTER Trigger

BEFORE	AFTER
Runs before the triggering operation is completed.	Runs after the triggering operation.
Can be useful for validation or preparing values, subject to DBMS rules.	Can be useful for auditing or actions based on the completed change.

10. Materialized View — Placement Level

Materialized view: Stores the result of a query physically and must be refreshed to reflect changes in underlying data. Unlike an ordinary view, it is not simply a virtual query result at read time.

11. Quick Comparison

Object	Main purpose
View	Virtual table / controlled query access.
Materialized View	Stored query result for faster reads in suitable workloads.
Procedure	Reusable set of database operations.
Function	Reusable logic that returns a value.
Trigger	Automatically executes because of a database event.

■ Placement memory

VIEW → virtual table | **PROCEDURE** → perform operations | **FUNCTION** → return value | **TRIGGER** → automatic event

FINAL 1-MINUTE REVISION — MODULES 18–21

Topic	Remember
Index	Extra data structure for faster lookup.
Composite Index	Index on multiple columns; leading-column order matters.
B+ Tree	Balanced, high fan-out tree; data entries at leaves; leaves support range traversal.
Hashing	Maps key to bucket; excellent for equality search.
Collision	Different keys map to same bucket.
Static Hashing	Fixed number of primary buckets.
Dynamic Hashing	Hash structure can grow/reorganize.
View	Virtual table based on a query.

Topic	Remember
Materialized View	Physically stored query result; refresh needed.
Procedure	Stored reusable operations.
Function	Stored logic that returns a value.
Trigger	Automatically fires on a database event.

Placement practice: Learn the differences in the comparison tables, then solve questions on index types, B+ tree properties, hashing collisions, and View vs Procedure vs Function vs Trigger. These are common terminology-based interview areas.